

COVER PAGE

Future Interns – Cyber Security Task Vulnerability Assessment Report.

Future Interns

Prepared By:

Bhavanidharan Sathiyaseelan

College

Paavai Engineering College

Department

B.E. Cyber Security

Acknowledgement

I would like to express my sincere gratitude to **Future Interns** for providing me with the opportunity to work on the **API Security Risk Analysis** project. This internship has enhanced my understanding of REST APIs, HTTP protocols, API security concepts, and the practical use of tools such as Postman and Browser Developer Tools for security assessment.

I extend my heartfelt thanks to my mentors and the Future Interns team for their continuous guidance, valuable resources, and encouragement throughout this project. Their support helped me gain practical experience in identifying API security risks, analyzing request and response headers, validating API responses, and documenting security findings professionally.

I am also grateful to **Paavai Engineering College**, the Department of Cyber Security, my faculty members, and my family for their constant motivation and support during the successful completion of this project.

Executive Summary

Application Programming Interfaces (APIs) play a critical role in modern web applications by enabling secure and efficient communication between clients and servers. However, improperly configured APIs can expose sensitive data and introduce security vulnerabilities. This project presents a comprehensive **API Security Risk Analysis** conducted on the **JSONPlaceholder REST API**, focusing on endpoint behavior, HTTP request and response headers, data exposure, and overall security posture.

The assessment was performed using **Postman**, **Firefox Browser**, and **Browser Developer Tools** to inspect API requests, validate responses, analyze HTTP headers, and identify potential security risks. The analysis included evaluating REST API endpoints, reviewing response structures, examining security-related headers, and verifying API behavior against common security best practices.

The findings indicate that the tested API functions correctly and provides consistent JSON responses while demonstrating standard RESTful communication. Several observations related to authentication, information disclosure, header configuration, and security hardening were documented, along with practical recommendations to improve API security. This report provides a structured overview of the assessment methodology, identified risks, and mitigation strategies, demonstrating the importance of secure API design and effective security testing in modern web applications.

Section	Page No.
Cover Page	1
Acknowledgement	2
Executive Summary	3
Table of Contents	4
1. Introduction	5
2. Objectives	6
3. Scope of Assessment	7
4. Tools Used	8
5. API Overview	9
6. Browser Developer Tools Analysis	13
7. Security Risk Assessment	14
8. Identified Findings	15
9. Recommendations	16
10. Conclusion	17
11. References	18
12. Appendix	19

1. Introduction

Application Programming Interfaces (APIs) are essential components of modern software systems, enabling seamless communication and data exchange between applications, web services, and devices. Most web and mobile applications rely on **RESTful APIs** to retrieve, process, and manage data efficiently using standard HTTP methods such as **GET**, **POST**, **PUT**, and **DELETE**.

As APIs often handle sensitive information and business-critical operations, they have become attractive targets for cyberattacks. Common API security risks include unauthorized access, broken authentication, excessive data exposure, improper input validation, security misconfigurations, and insufficient rate limiting. Identifying and mitigating these risks is essential to maintaining the confidentiality, integrity, and availability of application data.

This project focuses on conducting an **API Security Risk Analysis** using the **JSONPlaceholder REST API**, a publicly available API designed for learning and testing purposes. The assessment involves examining API endpoints, analyzing HTTP requests and responses, inspecting request and response headers, validating API behavior using **Postman**, and reviewing network traffic through **Browser Developer Tools**.

The primary objective of this analysis is to understand API communication, identify potential security risks, evaluate the implementation of security-related headers, and recommend best practices for improving API security. The findings presented in this report demonstrate practical API security assessment techniques and highlight the importance of secure API design in modern web applications.

2.Objectives

The primary objectives of this API Security Risk Analysis project are:

- To understand the architecture and functionality of RESTful APIs.
- To analyze API endpoints and evaluate their request and response behavior.
- To inspect HTTP request and response headers for security-related configurations.
- To validate API responses using Postman and Browser Developer Tools.
- To identify potential API security risks, including information disclosure and misconfigurations.
- To assess the implementation of standard HTTP methods and response status codes.
- To evaluate the exposure of API data and security-related headers.
- To recommend best practices and mitigation strategies for improving API security.
- To enhance practical knowledge of API testing and security assessment methodologies.
- To document the assessment process, findings, and recommendations in a professional technical report.

3.Scope of Assessment

This project focuses on evaluating the security posture of the **JSONPlaceholder REST API** through a structured analysis of its publicly accessible endpoints. The assessment is limited to non-intrusive and read-only testing performed in a controlled environment without modifying server-side data or attempting to exploit vulnerabilities.

The scope of the assessment includes:

- Analysis of REST API endpoints (/users, /users/{id}, /posts, and /photos).
- Inspection of HTTP request and response messages.
- Examination of HTTP request and response headers.
- Validation of API responses, status codes, and JSON data structures.
- Testing API functionality using **Postman**.
- Analysis of network traffic using **Firefox Browser Developer Tools**.
- Identification of potential security risks, including information disclosure, missing security controls, and header misconfigurations.
- Documentation of findings and recommendations based on API security best practices.

Out of Scope

The following activities were not included in this assessment:

- Authentication and authorization testing (API does not require authentication).
- Penetration testing or exploitation of vulnerabilities.
- Denial-of-Service (DoS) or stress testing.
- Source code review or server-side configuration analysis.
- Testing of private or production APIs.

4.Tools Used

The following tools were used to perform the API Security Risk Analysis:

Tool	Purpose
Postman	Sent API requests, validated responses, tested HTTP status codes, and verified API functionality.
Firefox Browser	Accessed API endpoints and viewed JSON responses directly in the browser.
Firefox Developer Tools (Network Inspector)	Inspected HTTP requests, response headers, request headers, status codes, and network traffic.
JSONPlaceholder REST API	Served as the target API for security analysis and endpoint testing.
JSON Viewer	Displayed JSON responses in a structured and readable tree format for easier analysis.

Tool Summary

- **Postman:** Used for sending GET requests, validating API responses, and executing test scripts.
- **Firefox Browser:** Used to access and verify API endpoints directly.
- **Firefox Developer Tools:** Used to analyze network requests, HTTP headers, response codes, and caching behavior.
- **JSONPlaceholder REST API:** Public REST API used as the testing environment.
- **JSON Viewer:** Assisted in visualizing and interpreting complex JSON response structures.

5. API Overview

The **JSONPlaceholder REST API** was selected as the target application for this security assessment because it is a publicly available API designed for learning, testing, and development purposes. It provides realistic sample data and supports standard RESTful operations, making it suitable for analyzing API behavior without affecting production systems.

The API communicates using the **HTTP protocol** and exchanges data in **JavaScript Object Notation (JSON)** format. It follows REST architecture by exposing resources through unique endpoints and responding to client requests with appropriate HTTP status codes and structured JSON data.

During this assessment, the API was examined to understand how requests are processed, how responses are generated, how HTTP headers are configured, and whether any potential security risks or information disclosures exist. The analysis focused on the behavior of publicly accessible endpoints and the security-related information contained within request and response messages.

5.1. API Endpoints Tested

The following REST API endpoints were analyzed during the security assessment:

Endpoint	HTTP Method	Purpose
/users	GET	Retrieves information about all users.
/users/1	GET	Retrieves detailed information about a specific user.
/posts	GET	Retrieves a collection of sample posts.
/photos	GET	Retrieves photo records and associated metadata.

Each endpoint was tested to verify:

- API accessibility
- HTTP status codes
- JSON response structure
- Response headers
- Request headers
- Data exposure
- API functionality

5.2. Testing Methodology

A structured methodology was followed to perform the API Security Risk Analysis. The assessment was conducted using non-intrusive testing techniques without modifying or exploiting the target API.

The testing process consisted of the following steps:

1. Identify the API endpoints to be tested.
2. Send HTTP GET requests using Postman.
3. Verify HTTP status codes and response times.
4. Inspect JSON response data for accuracy and completeness.
5. Analyze HTTP request headers transmitted by the client.
6. Examine HTTP response headers returned by the server.
7. Capture network traffic using Firefox Developer Tools.
8. Identify potential security risks and information disclosure.
9. Document observations, findings, and recommendations.

This methodology ensured a systematic evaluation of the API's functionality and security posture while adhering to ethical testing practices.

5.3. API Request & Response Analysis

The API communication process begins when the client sends an HTTP request to a specific endpoint. The server processes the request and returns an HTTP response containing a status code, response headers, and JSON-formatted data.

During the assessment, each endpoint returned a **200 OK** status, indicating successful request processing. The response bodies contained structured JSON objects and arrays representing users, posts, and photos. The API consistently delivered valid JSON responses with the appropriate **Content-Type: application/json** header.

The analysis confirmed that the API responses were well-structured, readable, and compliant with REST principles. No malformed responses or unexpected server errors were observed during testing.

Key observations include:

- Successful communication between client and server.
- Proper use of HTTP GET requests.
- Consistent JSON response format.
- Accurate HTTP status codes.
- Predictable REST API behavior.

- Publicly accessible endpoints without authentication.

5.4. HTTP Request Header Analysis

HTTP request headers contain information sent by the client to the server and provide details about the request environment.

The request headers observed during testing included:

Header	Purpose
Host	Specifies the target server.
User-Agent	Identifies the client application or browser.
Accept	Indicates the response formats accepted by the client.
Accept-Encoding	Specifies supported compression methods.
Accept-Language	Indicates the preferred language for responses.
Connection	Controls whether the network connection remains open.

The request headers appeared standard and contained no sensitive information such as authentication tokens or session cookies. This is expected because the tested API does not require user authentication.

5.5. HTTP Response Header Analysis

HTTP response headers provide metadata about the server response and reveal information related to content handling, caching, compression, and server configuration.

The following headers were identified during the assessment:

Header	Purpose
Content-Type	Specifies that the response is returned in JSON format.
Cache-Control	Defines browser and intermediary caching policies.
Connection	Indicates persistent HTTP connections.
Content-Encoding	Shows that the response is compressed.
ETag	Supports cache validation and conditional requests.
Server	Identifies the web server or reverse proxy.
Vary	Specifies how caches should differentiate responses.

The response headers demonstrated proper HTTP communication practices. However, the presence of certain headers, such as the **Server** header, may disclose implementation details that could assist attackers in fingerprinting the underlying infrastructure. Limiting unnecessary server information is considered a recommended security practice.

5.6. Postman Testing & Validation

Postman was used to send API requests, validate server responses, and verify the functionality of each endpoint.

The following activities were performed:

- Sent HTTP GET requests to each API endpoint.
- Verified successful **200 OK** status codes.
- Inspected JSON response bodies.
- Reviewed request and response headers.
- Measured response time and response size.
- Executed automated test scripts for response validation.

A simple validation script was used to confirm successful responses:

```
pm.test("Status code is 200", function () {  
    pm.response.to.have.status(200);  
});
```

The validation confirmed that all tested endpoints responded successfully with the expected status code. The API responses were consistent, properly formatted, and contained valid JSON data. Postman also provided detailed information about response headers, execution time, and payload size, enabling a comprehensive assessment of API behavior and reliability.

6. Browser Developer Tools Analysis

Browser Developer Tools were used to inspect the API requests and responses during testing. The **Network** tab was used to view HTTP requests, response headers, status codes, response time, and JSON data returned by the API.

The analysis confirmed that all tested endpoints responded successfully with a **200 OK** status code and returned data in **JSON** format. Request and response headers such as **Content-Type**, **Cache-Control**, **Connection**, and **Server** were also examined to understand how the API handled client requests.

Key Observations

- Inspected API requests using the Network tab.
- Verified **HTTP GET** requests.
- Confirmed **200 OK** status code.
- Examined request and response headers.
- Validated JSON response format.
- Reviewed caching and server information.

This analysis helped verify API communication and identify basic security-related information exposed through HTTP headers.

7. Security Risk Assessment

The API Security Risk Assessment was conducted by analyzing API endpoints, HTTP requests, responses, and headers to identify potential security issues. Since the **JSONPlaceholder API** is a public API designed for testing and learning, some security controls such as authentication are intentionally not implemented.

Risk Assessment

Security Risk	Severity	Observation
No Authentication	Medium	Public endpoints can be accessed without login.
Information Disclosure	Low	Server details are visible in response headers.
Public Data Exposure	Low	Sample data is publicly accessible.
Missing Security Headers	Low	Some recommended security headers are not present.
Rate Limiting Not Verified	Low	No visible rate-limiting information during testing.

Overall Assessment

The API performed as expected and returned valid responses without errors. No critical vulnerabilities were identified during the assessment. However, implementing authentication, limiting information disclosure, and adding appropriate security headers would improve the overall security of a production API.

8. Identified Findings

The following findings were observed during the API Security Risk Analysis:

- All tested API endpoints responded successfully with **HTTP 200 OK** status.
- API responses were returned in a valid **JSON** format.
- The **Content-Type** header was correctly set to application/json.
- Request and response headers were successfully captured and analyzed.
- Public API endpoints were accessible without authentication.
- Server information was visible in the HTTP response headers.
- Response caching was enabled through the **Cache-Control** header.
- No sensitive information, such as passwords or API keys, was exposed in the responses.
- The API demonstrated consistent and reliable performance during testing.
- No critical security vulnerabilities were identified during the assessment.

9. Recommendations

Based on the API Security Risk Analysis, the following recommendations are suggested to improve API security:

- Implement authentication and authorization mechanisms such as **API Keys**, **JWT**, or **OAuth 2.0**.
- Enable **HTTPS** for all API communications to protect data in transit.
- Add appropriate **security headers** to reduce information disclosure and improve protection.
- Limit unnecessary server information in HTTP response headers.
- Implement **rate limiting** to prevent abuse and denial-of-service attacks.
- Validate and sanitize all user inputs to prevent injection attacks.
- Return only the required data in API responses to minimize data exposure.
- Monitor and log API requests to detect suspicious activities.
- Perform regular security testing and vulnerability assessments.
- Follow the **OWASP API Security Top 10** best practices when developing and maintaining APIs.

Implementing these recommendations will strengthen the security of APIs, reduce potential risks, and improve the overall reliability and resilience of web applications.

10. Conclusion

This API Security Risk Analysis successfully evaluated the **JSONPlaceholder REST API** by examining its endpoints, HTTP requests, responses, and headers using **Postman**, **Firefox Browser**, and **Browser Developer Tools**. The assessment confirmed that the API responded correctly with valid JSON data and appropriate HTTP status codes.

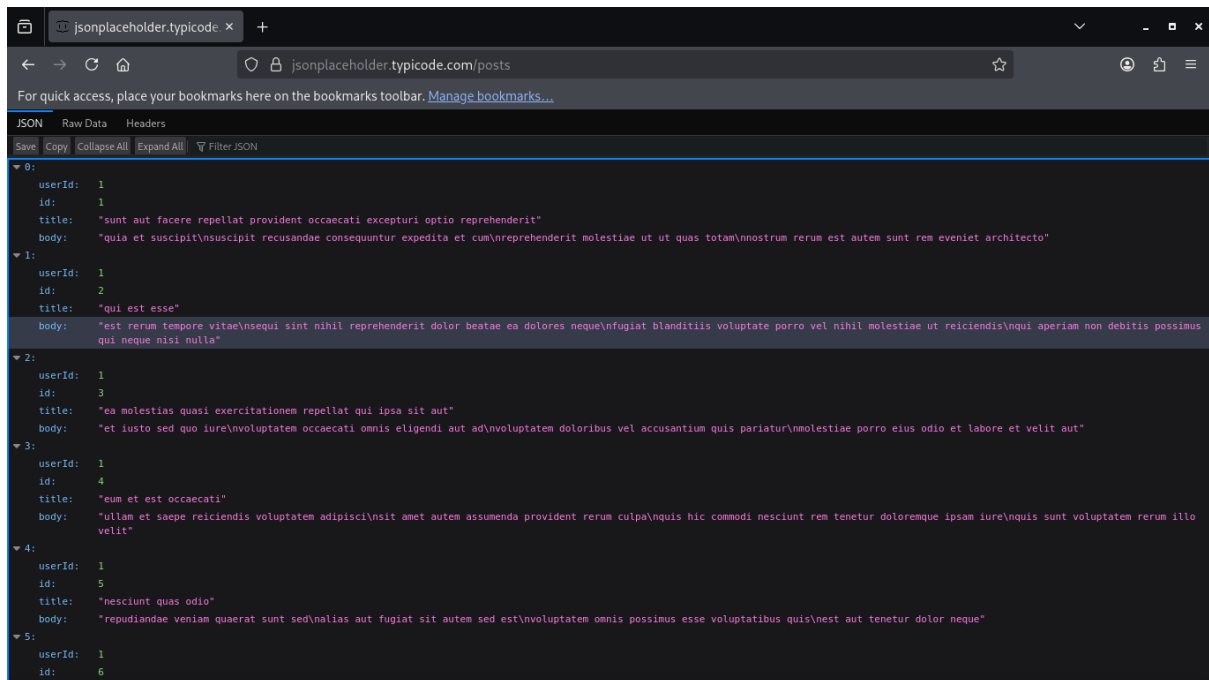
No critical security vulnerabilities were identified during the assessment. However, a few security improvements, such as implementing authentication, reducing information disclosure, adding security headers, and applying rate limiting, are recommended for production environments.

Overall, this project enhanced practical knowledge of REST APIs, API testing techniques, HTTP communication, and API security best practices. It also demonstrated the importance of regular API security assessments in maintaining secure and reliable web applications.

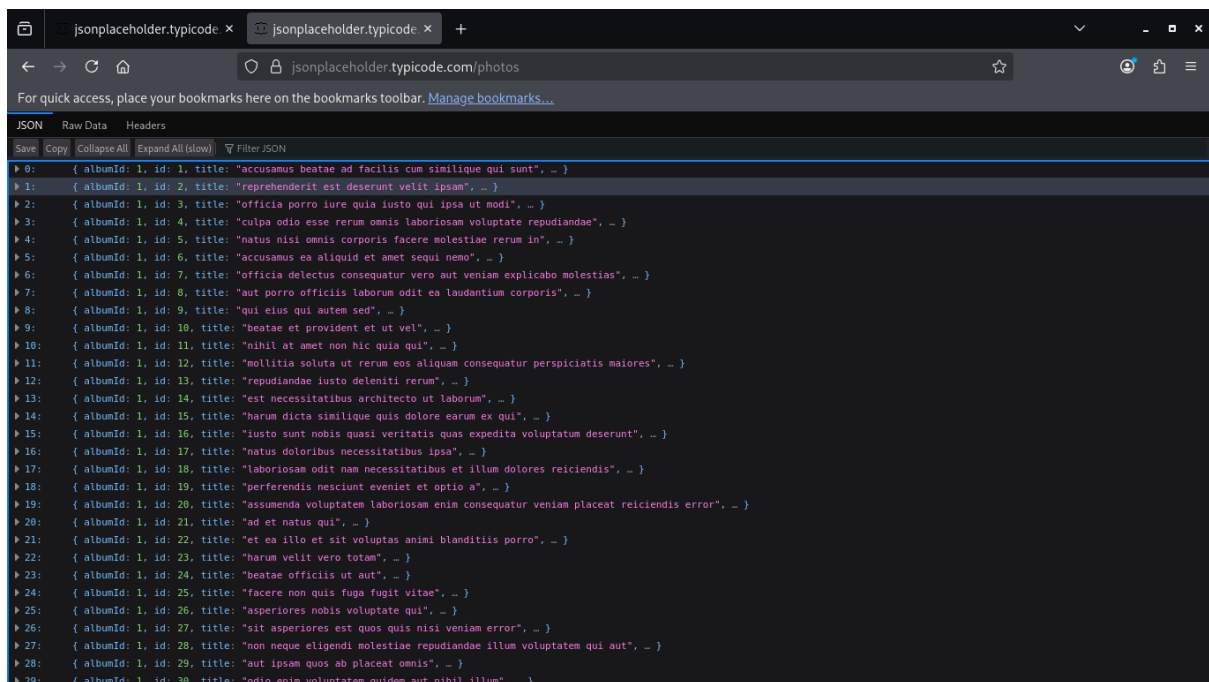
11. References

1. OWASP Foundation. *OWASP API Security Top 10*. <https://owasp.org/API-Security/>
2. JSONPlaceholder. *Fake Online REST API for Testing and Prototyping*. <https://jsonplaceholder.typicode.com/>
3. Postman. *Postman Learning Center*. <https://learning.postman.com/>
4. Mozilla Developer Network (MDN). *HTTP Overview*. <https://developer.mozilla.org/en-US/docs/Web/HTTP>
5. Mozilla Developer Network (MDN). *HTTP Headers*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
6. Mozilla Developer Network (MDN). *Firefox Developer Tools*. <https://firefox-source-docs.mozilla.org/devtools-user/>
7. RFC 9110. *HTTP Semantics*. Internet Engineering Task Force (IETF). <https://www.rfc-editor.org/rfc/rfc9110>
8. REST API Tutorial. *REST Architectural Principles*. <https://restfulapi.net/>
9. OpenAPI Initiative. *OpenAPI Specification*. <https://www.openapis.org/>
10. NIST. *Security and Privacy Controls for Information Systems and Organizations (SP 800-53 Rev. 5)*. <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final>

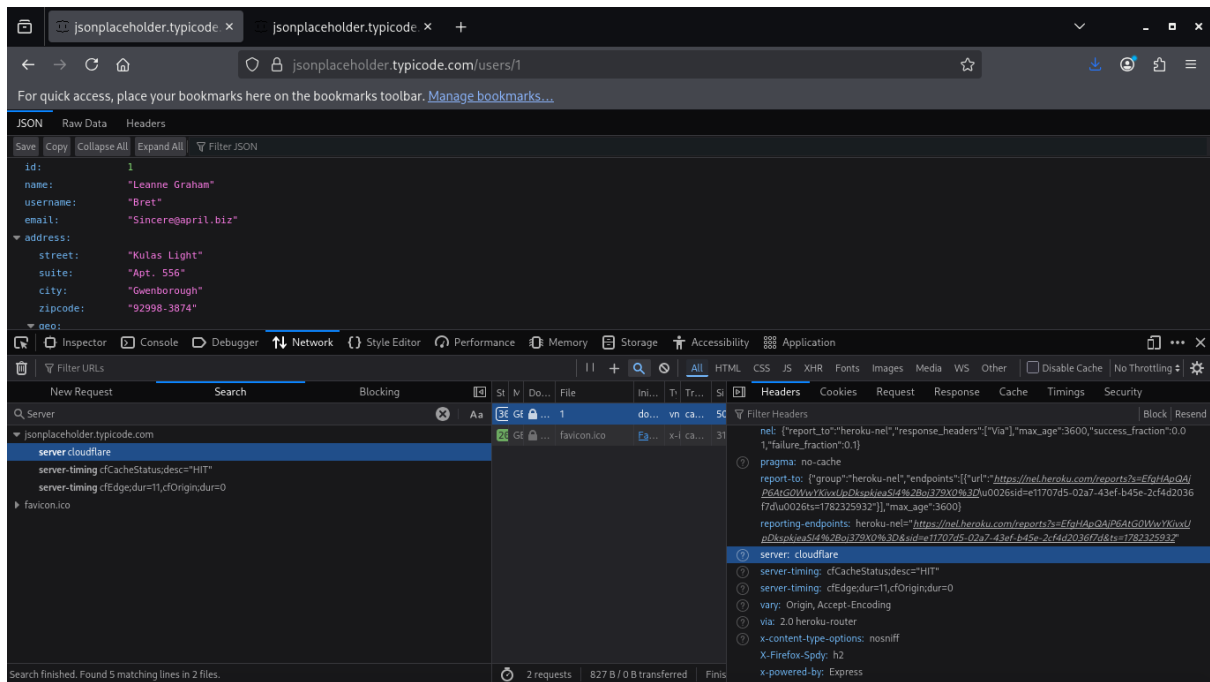
12.Appendix



01_JSONPlaceholder_Posts_JSON.png



Photos_API_JSON_Tree_View.png



Server_Header_Inspection.png

